Application-specific access policies using Zscaler Private Access (ZPA).

APPLICATION SPECIFIC ACCESS POLICIES USING ZSCALER PRIVATE ACCESS (ZPA)

Chandru. OP - chandruop2023aids@gmail.com

Dinesh. J - dineshj2023aids@gmail.com

Dinesh. R - dineshr2023aids@gmail.com

Jeeva Bharathi. S - jeevabharathis2023aids@gmail.com

JeevanandhaKumar. K – jeevanandhakumark2023aids@gmail.com

INTRODUCTION:

In today's digital world, organizations use many online systems, cloud services, and remote access technologies for their daily work. Employees often work from different locations and need to access company applications through the internet. Because of this, protecting organizational data and systems has become very important. At the same time, cyber-attacks and security threats are increasing rapidly. Hackers try to access sensitive information by exploiting weaknesses in traditional security systems.

Earlier, most organizations used **Virtual Private Network (VPN)** to allow employees to access internal applications from outside the company network. VPN allows users to connect to the organization's internal network through the internet. However, VPN usually provides access to a large part of the network once the user is connected. This increases the risk of unauthorized access and makes the system more vulnerable to cyber-attacks. If an attacker gains access to the network through VPN, they may be able to move inside the network and access other systems.

To overcome these problems, modern organizations are adopting a new security model called **Zero Trust Security Architecture**. The main idea of Zero Trust is **"Never Trust, Always Verify."** This means that no user or device is trusted automatically, even if they are inside the network. Every request to access an application must be verified before access is granted. This approach helps organizations improve their security and protect important data.

In this project, **Zscaler Private Access (ZPA)** is used to implement the Zero Trust security model. ZPA is a cloud-based service that provides secure access to internal applications without exposing the entire network. Instead of giving users access to the full network, ZPA connects users directly to the specific applications they are authorized to use. This connection is created through a secure channel, ensuring that the communication between the user and the application remains protected.

Another important advantage of ZPA is that it hides the internal applications from the public internet. Only authenticated and authorized users can see and access the applications. The system checks the user's identity, device status, and security policies before allowing access. This process reduces the chances of unauthorized access and protects the organization's internal resources.

Therefore, this project focuses on implementing secure application access using **Zscaler Private Access** based on the **Zero Trust model**. The main aim is to improve security, reduce network risks, and

provide safe access to internal applications for authorized users. By using this approach, organizations can protect their systems from cyber threats and maintain a more secure digital environment.

ABSTRACT:

In the modern digital environment, many organizations use cloud services and remote access to perform their daily operations. Employees often work from different locations and need secure access to company applications through the internet. However, traditional security methods such as Virtual Private Networks (VPNs) may create security risks because they allow users to access the internal network directly. This increases the possibility of cyber-attacks and unauthorized access.

This project focuses on implementing Zero Trust security architecture using Zscaler Private Access (ZPA). The main objective of this system is to provide secure access to internal applications without exposing the entire network. The Zero Trust model follows the principle “Never Trust, Always Verify,” which means every user and device must be verified before accessing any application.

ZPA allows users to connect only to the applications they are authorized to use. It creates a secure connection between the user and the application without giving access to the whole network. The system checks the user identity, device security, and access policies before allowing the connection.

By using this approach, organizations can reduce security risks, prevent unauthorized access, and protect sensitive data. This project shows how Zero Trust architecture with Zscaler Private Access can improve network security and provide safe access to applications in modern organizations.

Use Case Scenarios

Scenario 1: Employee Access to Internal Application

An employee in the company wants to access an internal application from home. The employee opens the ZPA Client Connector and logs in using the company username and password.

The system checks the user identity and verifies the device security. If the user is authorized, ZPA creates a secure connection between the user and the required application. The employee can access only that application and cannot see other internal systems.

Output:

- User identity is verified
- Secure connection is created to the required application
- Other internal applications are hidden

Scenario 2: Department Based Application Access

In an organization, different departments use different applications. For example, the finance department uses a finance application and the HR department uses an HR application.

With ZPA, employees can only access the applications related to their department. The system verifies the user role and then provides access to the correct application. This helps to maintain security and prevents unauthorized access.

Output:

- Access is given based on department role
- Only required application is accessible
- Unauthorized access is prevented

Scenario 3: Remote Employee Secure Access

Sometimes employees work from different locations such as home or while travelling. They need secure access to company applications.

The employee logs in through the ZPA client. The system verifies the user credentials and device security. After verification, ZPA provides secure access to the required application without giving access to the entire network.

Output:

- Secure remote access is provided
- User identity and device are verified
- Internal network remains protected

Scenario 4: Administrator Managing Access

The system administrator manages application access in the organization. The administrator defines security policies and decides which user can access which application.

If any user tries to access an unauthorized application, the system blocks the request. This helps maintain security and control within the organization.

Output:

- Administrator controls user access
- Security policies are applied
- Unauthorized access is blocked

Scenario 5: Secure Access to Internal Files

In an organization, some important files are stored in internal servers. Only authorized employees should access these files. An employee who needs these files logs in to the system using the ZPA Client Connector.

The system checks the user identity and verifies whether the employee has permission to access the files. If the user is authorized, ZPA creates a secure connection to the file server. The employee can view or download the required files safely.

The user cannot access other servers or applications that are not allowed. This helps to protect important company information from unauthorized access.

Output:

- User identity is verified before access
- Secure connection is created to the file server
- Only authorized files can be accessed
- Other systems remain hidden and protected

Stage-1

Title:

Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Overview:

In modern organizations, technology plays an important role in daily operations. Employees need to access company applications, data, and services from different locations such as offices, homes, or while travelling. Because of this, organizations must ensure that their systems and data are protected from cyber threats and unauthorized access.

Earlier, many organizations used Virtual Private Network (VPN) technology to allow employees to access internal applications remotely. VPN connects the user to the company network through the internet. However, once a user connects through VPN, they may get access to many parts of the network even if they only need one application. This increases security risks and creates more opportunities for cyber attackers.

To overcome these security problems, organizations are now adopting a modern security approach called Zero Trust Security Architecture. The main principle of Zero Trust is “Never Trust, Always Verify.” This means that every user and device must be verified before they are allowed to access any application or system. The system does not automatically trust any user, even if the user is inside the organization’s network.

This project focuses on implementing Zscaler Private Access (ZPA) to provide secure access to internal applications. ZPA is a cloud-based security service that helps organizations control how users access applications. Instead of allowing users to enter the entire network, ZPA connects the user directly to the specific application they are allowed to use.

When a user tries to access an application, the system first verifies the user identity using authentication methods. It also checks the security status of the user’s device. After successful verification, ZPA creates a secure connection between the user and the required application. This connection ensures that the communication is safe and protected from external threats.

Another important advantage of ZPA is that internal applications remain hidden from the public internet. Only authorized users can see and access these applications. This reduces the chances of cyber-attacks and helps organizations protect their sensitive data.

Overall, this project explains how Zero Trust Architecture with Zscaler Private Access can improve network security, reduce risks, and provide secure application access for users working from different locations.

List of teammates-

S.no	Name	College	Contact
1.	Chandru.OP	Shree Venkateshwara Arts and Science(Co-Ed) College	Chandruop2023aids@gmail.com
2.	Dinesh.J	Shree Venkateshwara Arts and Science(Co-Ed) College	Dineshj2023aids@gmail.com
3	Dinesh.R	Shree Venkateshwara Arts and Science(Co-Ed) College	Dineshr2023aids@gmail.com
4	JeevaBharathi.S	Shree Venkateshwara Arts and Science(Co-Ed) College	Jeevabharathis2023aids@gmail.com
5	JeevanandhaKumar.k	Shree Venkateshwara Arts and Science(Co-Ed) College	Jeevanandhakumark2023aids@gmail.com

List of vulnerability table-

S.NO	CATEGORY NAME	VULNERABILITY NAME	CWE/CVE-NO
1.	Broken Access Control	Exposure of Sensitive information to an unauthorized actor	CWE-200
2.	Security Misconfiguration	Active Debug Code	CWE-489
3.	Software Supply Chain Failures	Reliance on Component That is Not Updatable	CWE-1329

4.	Cryptographic Failures	Key Exchange Without Entity Authentication	CWE-322
5.	Injection	Improper Neutralization of HTTP Headers For Scripting Syntax	CWE-644
6.	Insecure Design	Trust Boundary Violation	CWE-501
7.	Authentication Failures	Authentication Bypass by Alternate Name	CWE-289
8.	Software or Data Integrity Failures	Insufficient Verification of Data Authenticity	CWE-345
9.	Security Logging and Alerting Failures	Information Loss or Omission	CWE-221
10.	Mishandling of Exceptional Conditions	NULL Pointer Dereference	CWE-476

REPORT

1. Vulnerability Name

Exposure of Sensitive Information to an Unauthorize Author

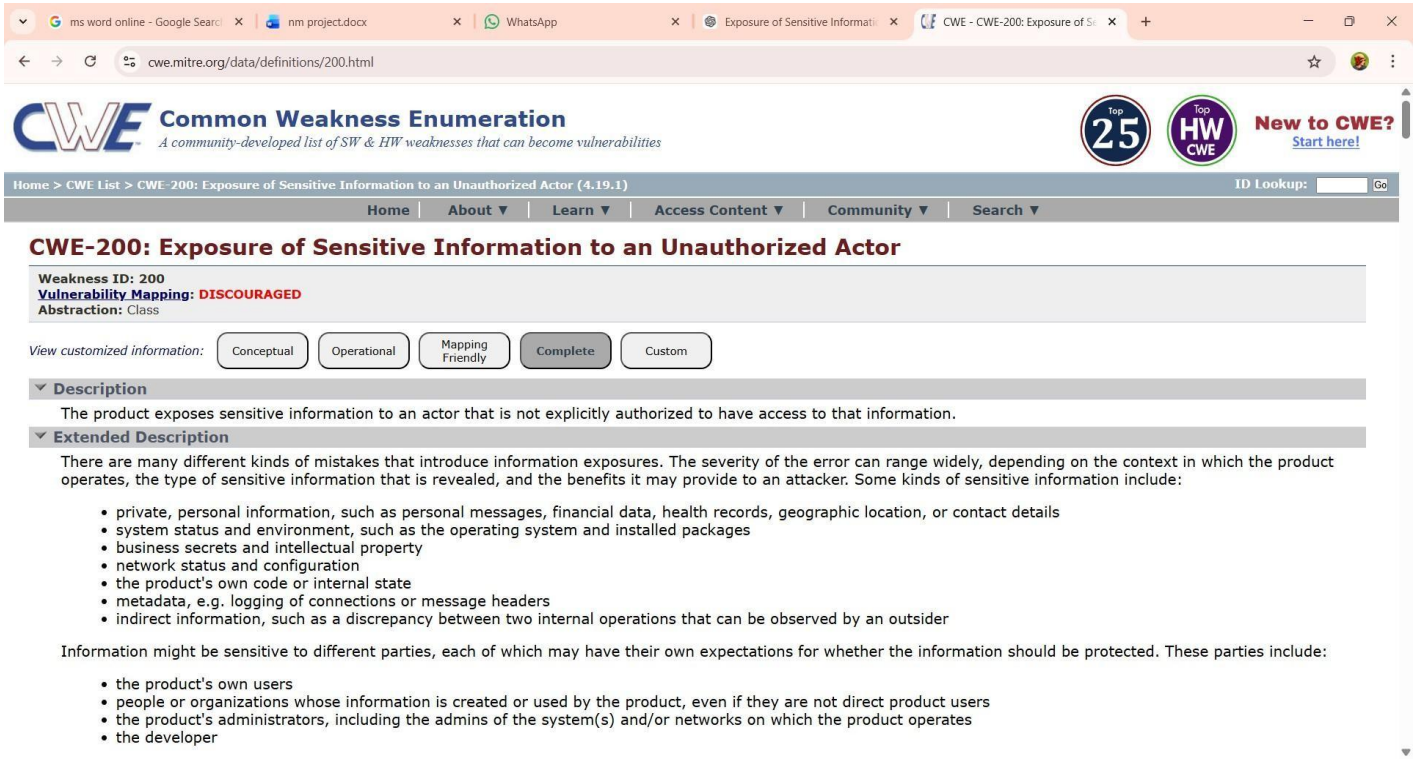
OWASP Category :OWASP Top 10-Broken Access Control

Description:

Exposure of Sensitive Information to an Unauthorized Author refers to a security vulnerability where confidential or private data is accessed, viewed, or modified by a person who does not have proper permission. This usually happens due to weak access control, improper authentication mechanisms, misconfigured systems, or insecure APIs. Sensitive information may include user credentials, personal data, financial details, business documents, or system configuration data. When authorization controls are not properly implemented, unauthorized users can gain access to this data, leading to privacy and security risks.

Business Impact:

The exposure of sensitive information can cause serious damage to an organization. It may lead to loss of customer trust, financial losses, and legal penalties due to violation of data protection regulations. Competitors may gain access to confidential business data, affecting the company's competitive advantage. In addition, attackers may use the exposed data for fraud, identity theft, or further cyberattacks on the system. Overall, such vulnerabilities can harm the organization's reputation, operational stability, and long-term business growth.



cwe.mitre.org/data/definitions/200.html

Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-200: Exposure of Sensitive Information to an Unauthorized Actor (4.19.1)

ID Lookup:

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

Weakness ID: 200
Vulnerability Mapping: DISCOURAGED
Abstraction: Class

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product exposes sensitive information to an actor that is not explicitly authorized to have access to that information.

Extended Description

There are many different kinds of mistakes that introduce information exposures. The severity of the error can range widely, depending on the context in which the product operates, the type of sensitive information that is revealed, and the benefits it may provide to an attacker. Some kinds of sensitive information include:

- private, personal information, such as personal messages, financial data, health records, geographic location, or contact details
- system status and environment, such as the operating system and installed packages
- business secrets and intellectual property
- network status and configuration
- the product's own code or internal state
- metadata, e.g. logging of connections or message headers
- indirect information, such as a discrepancy between two internal operations that can be observed by an outsider

Information might be sensitive to different parties, each of which may have their own expectations for whether the information should be protected. These parties include:

- the product's own users
- people or organizations whose information is created or used by the product, even if they are not direct product users
- the product's administrators, including the admins of the system(s) and/or networks on which the product operates
- the developer

Recommendation/Mitigation:

1. ImplementStrongAccessControl

Use proper authentication and authorization mechanisms to ensure that only authorized users can access sensitive information. Apply role-based access control (RBAC) so users can access only the data required for their role.

2. Follow the Principle of Least Privilege

Provide users with the minimum level of access necessary to perform their tasks. This reduces the risk of sensitive data being accessed by unauthorized individuals.

3. Encrypt Sensitive Data

Protect sensitive information using strong encryption techniques both in transit (HTTPS/SSL) and at rest to prevent unauthorized access.

4. Use Secure Authentication Methods

Implement strong password policies and multi-factor authentication (MFA) to add an extra layer of security.

5. **Regular Security Audits and Monitoring**

Conduct regular vulnerability scans and security audits to identify misconfigurations or unauthorized access attempts. Monitor system logs to detect suspicious activities.

6. **Proper Configuration of Systems and APIs**

Ensure that applications, servers, and APIs are securely configured and do not expose sensitive data through misconfigured endpoints or error messages.

7. **Data Masking and Tokenization**

Mask or tokenize sensitive information such as passwords, personal data, or financial details so that even if accessed, the information cannot be misused.

8. **Security Awareness and Training**

Train employees and developers on secure coding practices and data protection policies to prevent accidental exposure of sensitive information.

2. **Vulnerability Name:** Active Debug Code

CWE/CVE: CWE-489

OWASP/SANS Category: OWASP Top 10- Security Misconfiguration

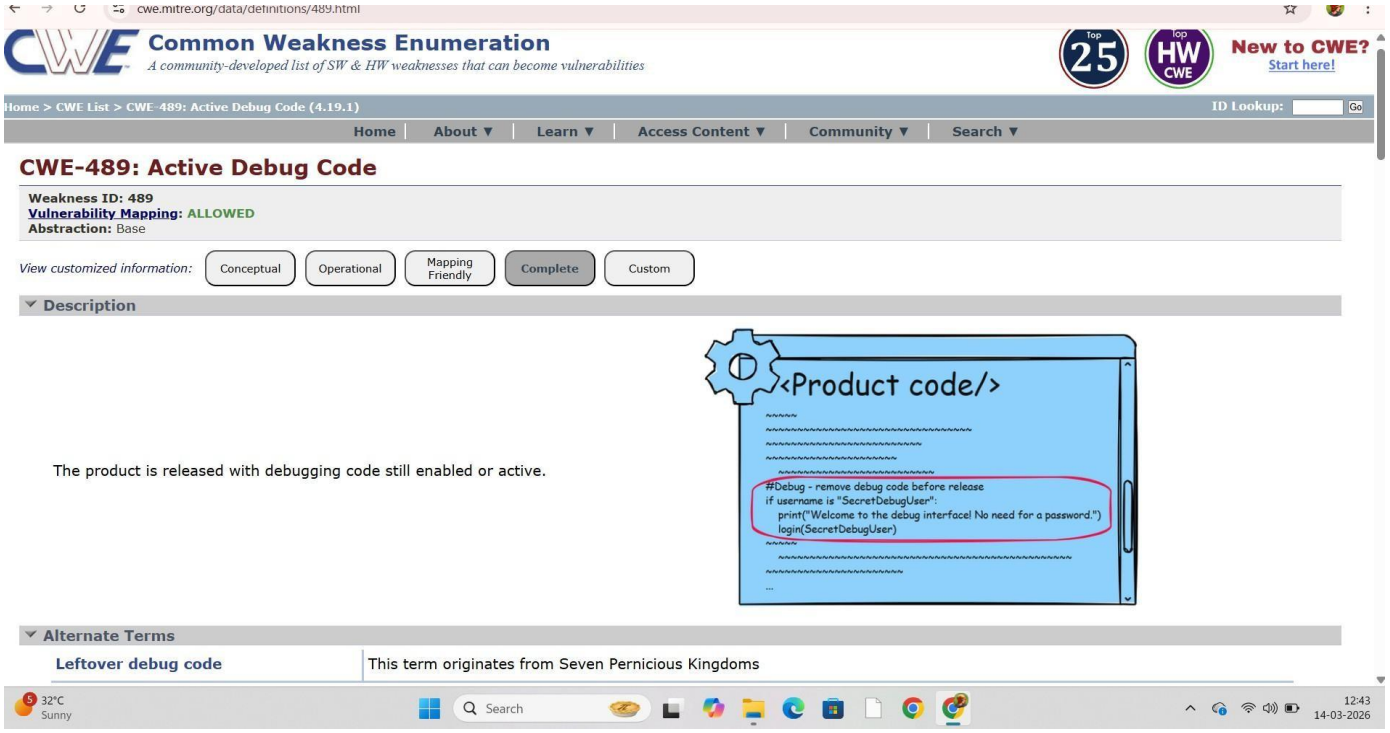
Description:

Active Debug Code refers to debugging statements or testing code that remains in the application after development and deployment. Developers often use debug code to test functionality, track errors, or print internal system information during the development stage. If this code is not removed before the application is released, it may expose sensitive system details such as file paths, database queries, user data, configuration settings, or system errors. Attackers can use this information to understand the system structure and exploit security weaknesses.

BusinessImpact:

The presence of active debug code can lead to security risks for the organization. It may reveal sensitive information about the application, making it easier for attackers to find vulnerabilities and launch cyberattacks. This can result in data breaches, system compromise, and unauthorized access to confidential information. In addition, such security weaknesses can damage

the company's reputation, reduce customer trust, and may lead to financial losses or regulatory penalties if sensitive data is exposed.



Recommendation/Mitigation:

1. Remove Debug Code Before Production

Ensure that all debugging statements, test code, and development logs are removed before deploying the application to the production environment.

2. Use Production Configuration Settings

Configure the application to run in production mode, where debugging features and detailed error messages are automatically disabled.

3. Implement Secure Error Handling

Display only generic error messages to users and avoid revealing technical details such as system paths, database queries, or server information.

4. Conduct Code Reviews

Perform regular code reviews to identify and remove any remaining debug statements or unnecessary test code before release.

5. Perform Security Testing

Use vulnerability scanning and penetration testing to detect exposed debug information or insecure configurations.

6. Restrict Access to Logs

Ensure that application logs and debugging information are accessible only to authorized administrators and are stored securely.

7. Follow Secure Development Practices

should follow secure coding guidelines and maintain separate environments for development, testing, and production to prevent debug code from being deployed.

3. **Vulnerability Name:** Reliance on Component That is Not Updateable

CWE/CVE: 1329

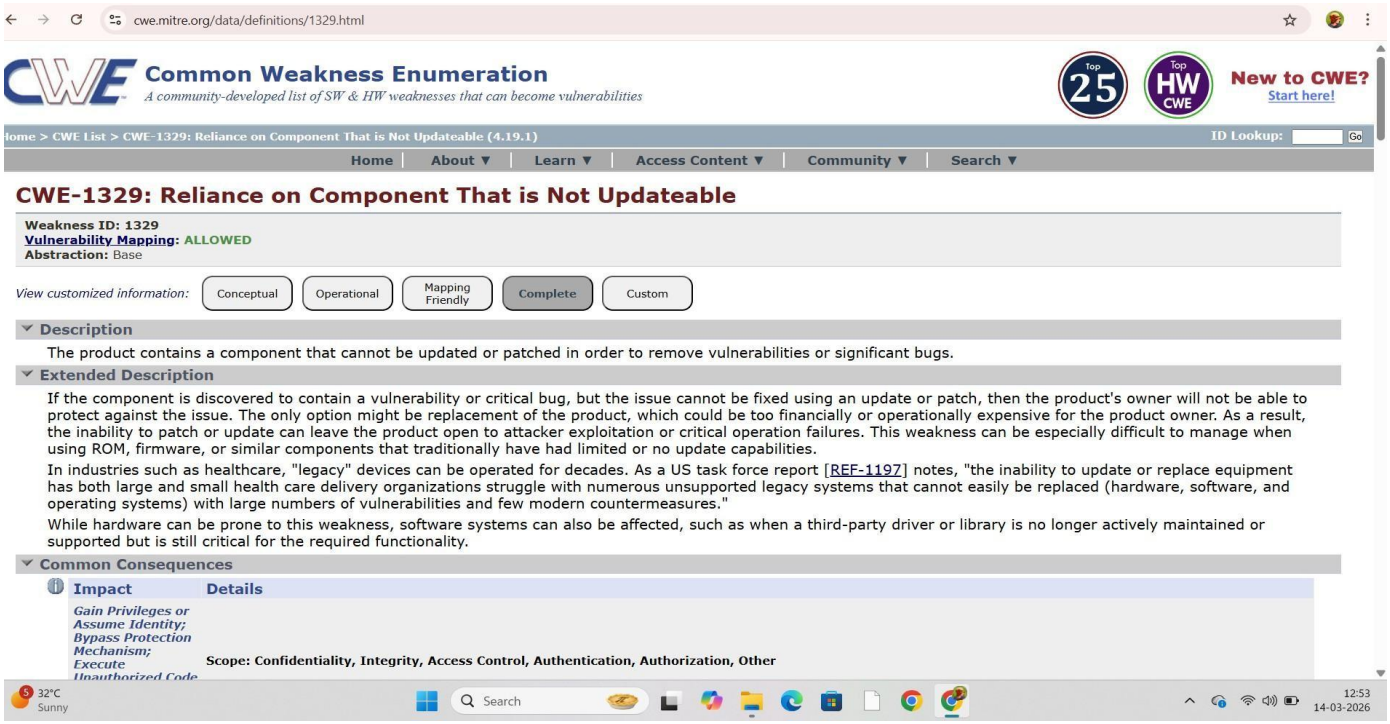
OWASP/SANS Category: OWASP Top 10- Software Supply Chain Failures

Description:

Reliance on Component That is Not Updateable refers to the use of software components, libraries, or third-party modules that cannot be updated or patched. These components may be outdated, unsupported, or no longer maintained by the vendor. If security vulnerabilities are discovered in such components, they cannot be fixed through updates. As a result, the application continues to run with known security weaknesses, making it more vulnerable to cyberattacks.

Business Impact:

Using components that cannot be updated can expose the organization to serious security risks. Attackers may exploit known vulnerabilities in outdated components to gain unauthorized access, steal sensitive data, or disrupt system operations. This may lead to data breaches, financial losses, and service downtime. Additionally, relying on unsupported components can create compliance issues with security standards and reduce the overall reliability and performance of the system. Over time, it may also increase maintenance costs and damage the organization’s reputation.



cwe.mitre.org/data/definitions/1329.html

Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities

Top 25 | Top HW CWE | New to CWE? Start here!

Home > CWE List > CWE-1329: Reliance on Component That is Not Updateable (4.19.1) | ID Lookup: Go

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-1329: Reliance on Component That is Not Updateable

Weakness ID: 1329
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product contains a component that cannot be updated or patched in order to remove vulnerabilities or significant bugs.

Extended Description

If the component is discovered to contain a vulnerability or critical bug, but the issue cannot be fixed using an update or patch, then the product's owner will not be able to protect against the issue. The only option might be replacement of the product, which could be too financially or operationally expensive for the product owner. As a result, the inability to patch or update can leave the product open to attacker exploitation or critical operation failures. This weakness can be especially difficult to manage when using ROM, firmware, or similar components that traditionally have had limited or no update capabilities.

In industries such as healthcare, "legacy" devices can be operated for decades. As a US task force report [REF-1197] notes, "the inability to update or replace equipment has both large and small health care delivery organizations struggle with numerous unsupported legacy systems that cannot easily be replaced (hardware, software, and operating systems) with large numbers of vulnerabilities and few modern countermeasures."

While hardware can be prone to this weakness, software systems can also be affected, such as when a third-party driver or library is no longer actively maintained or supported but is still critical for the required functionality.

Common Consequences

Impact	Details
Gain Privileges or Assume Identity; Bypass Protection Mechanism; Execute Unauthorized Code	Scope: Confidentiality, Integrity, Access Control, Authentication, Authorization, Other

32°C Sunny | Search | 12:53 14-03-2026

Recommendation/Mitigation:

1. Replace Outdated Components

Identify components that cannot be updated and replace them with modern, supported, and regularly maintained alternatives.

2. Use Regular Software Updates

Ensure all third-party libraries, frameworks, and dependencies are updated regularly to the latest secure versions.

3. Monitor Vulnerabilities

Use vulnerability scanning tools to monitor components for known security issues and take immediate action if vulnerabilities are found.

4. Choose Actively Maintained Components

When selecting third-party components, prefer those that are actively supported and frequently updated by developers or vendors.

5. Maintain a Software Inventory

Keep a detailed list of all software components and dependencies used in the application to track updates and security patches easily.

6. Implement Security Testing

Perform regular security assessments, code reviews, and penetration testing to identify risks caused by outdated or unsupported components.

7. Plan for Component Replacement

Create a long-term strategy to gradually phase out unsupported components and migrate to secure technologies.

4. Vulnerability Name: Key Exchange Without Entity Authentication

CWE/CVE: CWE-322

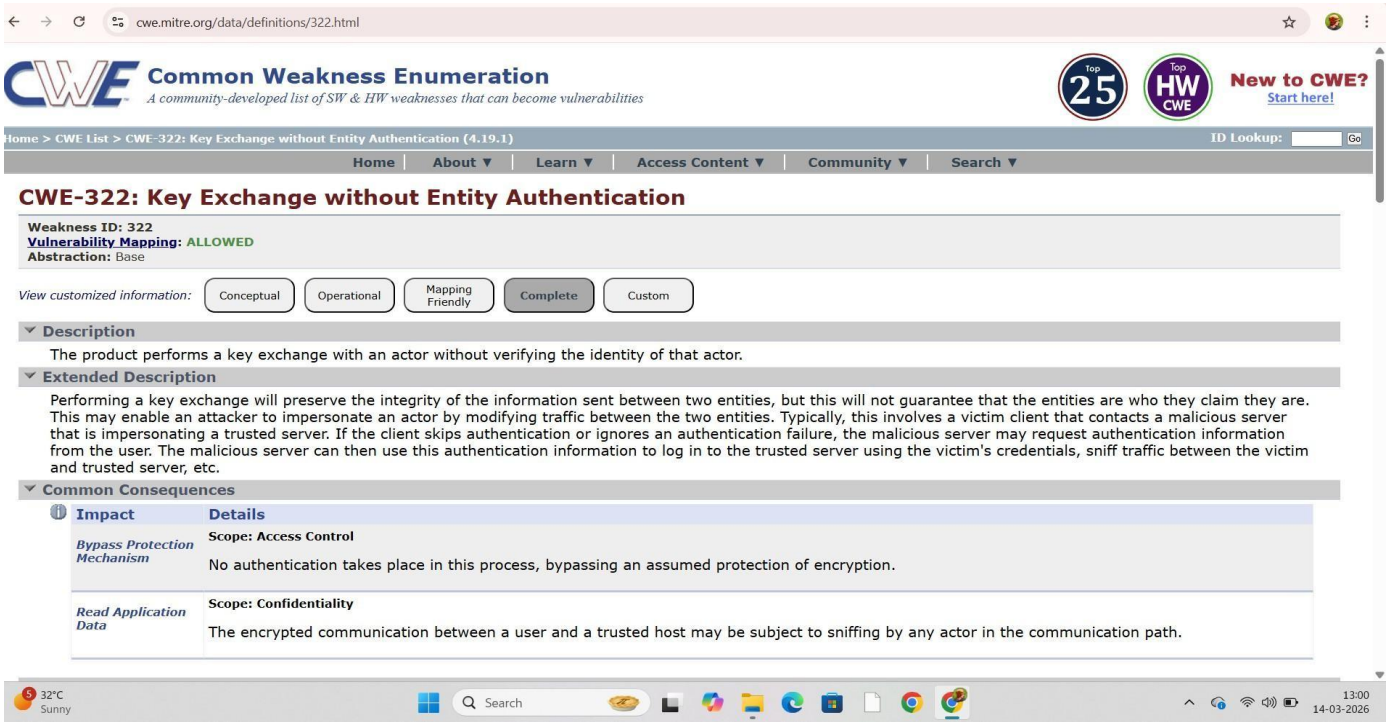
OWASP/SANS Category: OWASP Top 10- Cryptographic Failures

Description:

Key Exchange Without Entity Authentication refers to a security weakness where cryptographic keys are exchanged between systems or users without verifying the identity of the communicating parties. In such cases, the system establishes an encrypted connection, but it does not confirm whether the other party is legitimate or an attacker. This lack of authentication can allow attackers to intercept the communication and perform attacks such as man-in-the-middle (MITM), where the attacker secretly relays or alters the communication between two parties.

Business Impact:

This vulnerability can lead to serious security threats for an organization. Attackers may intercept sensitive data such as login credentials, financial information, or confidential business data during transmission. It can result in data breaches, unauthorized access to systems, and manipulation of critical information. Additionally, such incidents may cause financial losses, legal consequences, loss of customer trust, and damage to the organization's reputation.



cwe.mitre.org/data/definitions/322.html

Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-322: Key Exchange without Entity Authentication (4.19.1)

ID Lookup:

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-322: Key Exchange without Entity Authentication

Weakness ID: 322
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product performs a key exchange with an actor without verifying the identity of that actor.

Extended Description

Performing a key exchange will preserve the integrity of the information sent between two entities, but this will not guarantee that the entities are who they claim they are. This may enable an attacker to impersonate an actor by modifying traffic between the two entities. Typically, this involves a victim client that contacts a malicious server that is impersonating a trusted server. If the client skips authentication or ignores an authentication failure, the malicious server may request authentication information from the user. The malicious server can then use this authentication information to log in to the trusted server using the victim's credentials, sniff traffic between the victim and trusted server, etc.

Common Consequences

Impact	Details
Bypass Protection Mechanism	Scope: Access Control No authentication takes place in this process, bypassing an assumed protection of encryption.
Read Application Data	Scope: Confidentiality The encrypted communication between a user and a trusted host may be subject to sniffing by any actor in the communication path.

32°C Sunny | Search | 13:00 14-03-2026

Recommendation/Mitigation:

1. Implement Strong Authentication Mechanisms

Ensure that both communicating parties verify each other's identity before exchanging cryptographic keys. Use authentication methods such as digital certificates or secure authentication protocols.

2. Use Secure Communication Protocols

Adopt secure protocols like TLS (Transport Layer Security) that support authenticated key exchange and protect data during transmission.

3. Use Digital Certificates

Implement certificate-based authentication using trusted certificate authorities (CA) to verify the identity of servers and clients.

4. Enable Mutual Authentication

Use **mutual authentication** where both the client and server verify each other's identity during the key exchange process.

5. Protect Against Man-in-the-Middle Attacks

Apply security controls such as certificate validation and secure key exchange algorithms to prevent interception or tampering during communication.

6. Regular Security Monitoring and Testing

Conduct regular security assessments, vulnerability scans, and penetration testing to detect and fix weak authentication mechanisms.

5. Vulnerability Name: UNIX Symbolic Link (SymLink) Following

CWE:CWE-59: Improper Link Resolution Before File Access

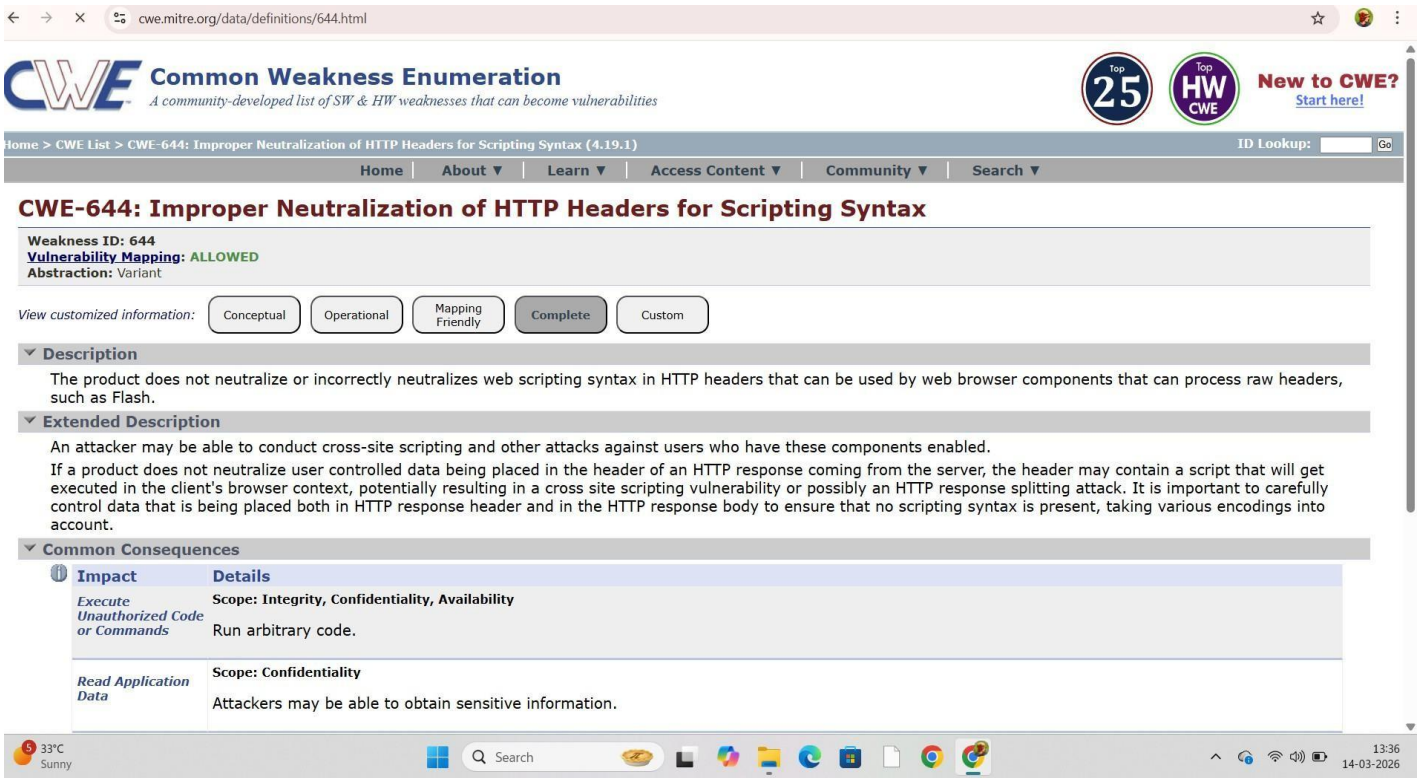
OWASP Category: A5: 2021- Security Misconfiguration / A01:2021- Broken Access Control

Description:

UNIX Symbolic Link (SymLink) Following is a vulnerability that occurs when an application follows a symbolic link (shortcut file) without properly verifying its target. In UNIX-based systems, a symbolic link is a file that points to another file or directory. If an attacker creates a malicious symbolic link and the application follows it without validation, the application may access or modify unintended files. This can allow attackers to read sensitive data, overwrite important system files, or gain unauthorized access.

Business Impact:

This vulnerability can lead to serious security risks for an organization. Attackers may exploit symbolic links to access confidential files, modify critical system data, or escalate privileges. It can result in data breaches, system compromise, or disruption of services. Additionally, such incidents may cause financial losses, legal issues, and damage to the organization's reputation due to unauthorized access to sensitive information.



cwe.mitre.org/data/definitions/644.html

Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-644: Improper Neutralization of HTTP Headers for Scripting Syntax (4.19.1)

ID Lookup:

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-644: Improper Neutralization of HTTP Headers for Scripting Syntax

Weakness ID: 644
Vulnerability Mapping: ALLOWED
Abstraction: Variant

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product does not neutralize or incorrectly neutralizes web scripting syntax in HTTP headers that can be used by web browser components that can process raw headers, such as Flash.

Extended Description

An attacker may be able to conduct cross-site scripting and other attacks against users who have these components enabled.

If a product does not neutralize user controlled data being placed in the header of an HTTP response coming from the server, the header may contain a script that will get executed in the client's browser context, potentially resulting in a cross site scripting vulnerability or possibly an HTTP response splitting attack. It is important to carefully control data that is being placed both in HTTP response header and in the HTTP response body to ensure that no scripting syntax is present, taking various encodings into account.

Common Consequences

Impact	Details
Execute Unauthorized Code or Commands	Scope: Integrity, Confidentiality, Availability Run arbitrary code.
Read Application Data	Scope: Confidentiality Attackers may be able to obtain sensitive information.

33°C Sunny | Search | 13:36 14-03-2026

Recommendation/Mitigation:

1. Validate File Paths

Ensure the application properly validates file paths before accessing files. Avoid blindly following symbolic links without verifying the actual target file.

2. Use Secure File Handling Methods

Use secure system calls and file-handling techniques that prevent automatic following of symbolic links.

3. Implement Proper Access Control

Restrict file and directory permissions so that unauthorized users cannot create or modify symbolic links in sensitive directories.

4. Avoid Using Shared Temporary Directories

Do not store sensitive files in publicly writable directories such as `/tmp`. Use secure directories with restricted permissions.

5. Check File Ownership and Permissions

Before accessing a file, verify that the file owner and permissions are valid and trusted.

6. Regular Security Audits

Perform regular security testing and vulnerability scanning to detect symbolic link related issues in the system.

6.Vulnerability Name: Windows Hard Link Following

CWE: CWE-59: Improper Link Resolution Before File Access

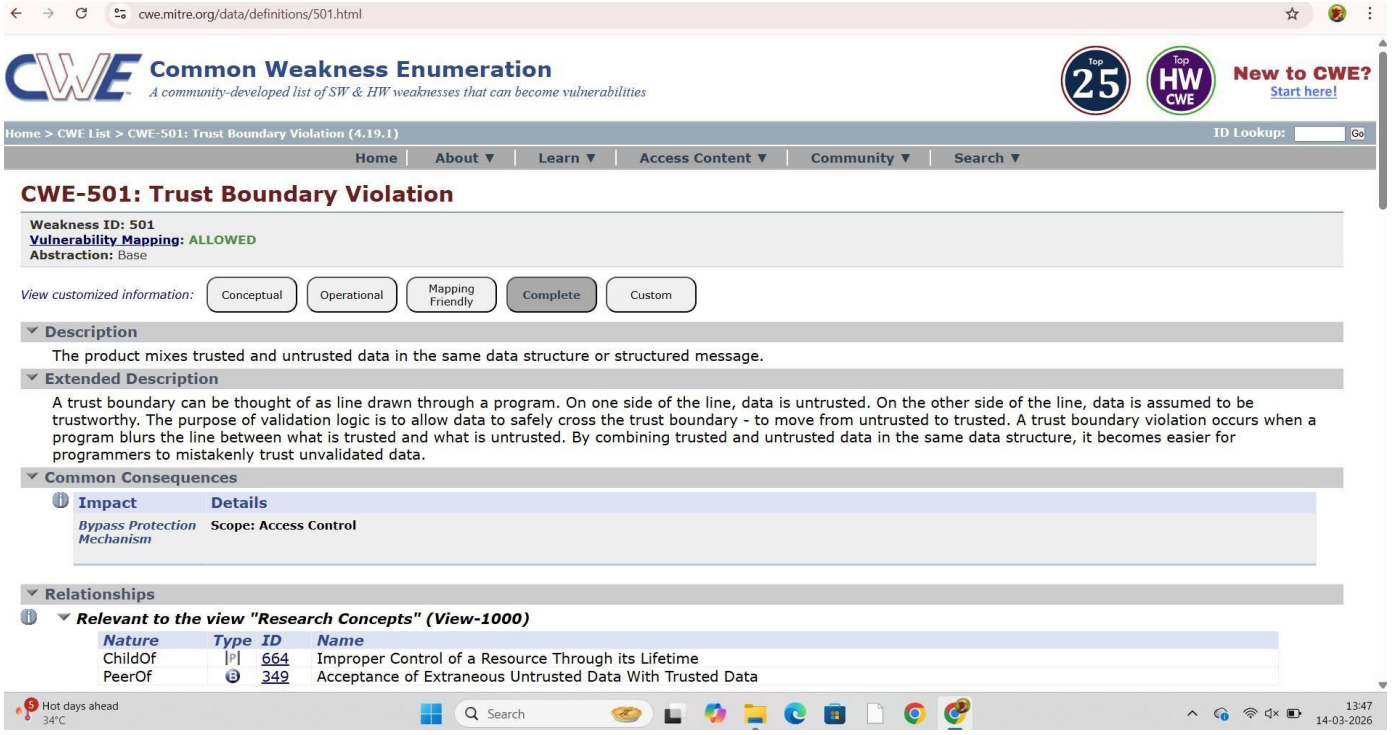
OWASP Category: A5:2021- Security Misconfiguration / A01: 2021- Broken Access Control

Description:

Windows Hard Link Following is a security vulnerability that occurs when an application follows a hard link without properly verifying the file it points to. In Windows systems, a hard link is a file that references the same physical data on the disk as another file. If an attacker creates a malicious hard link that points to a sensitive system file, the application may unknowingly read, modify, or overwrite that file when it accesses the hard link. This happens when the application does not properly validate the file path or permissions.

Business Impact:

This vulnerability can allow attackers to manipulate or overwrite critical system files, leading to unauthorized access or system compromise. Attackers may exploit this weakness to gain higher privileges, modify sensitive data, or disrupt system operations. Such incidents can cause data breaches, financial losses, and service interruptions. Additionally, it may damage the organization's reputation and reduce customer trust due to security failures.



Recommendation/Mitigation:

1. Validate File Paths

Applications should verify the actual file path and ensure that it points to the intended file before performing any read or write operations.

2. Restrict File Permissions

Limit the ability of normal users to create hard links in sensitive directories by applying proper file system permissions and access controls.

3. Use Secure File Handling Techniques

Developers should implement secure file-handling methods that prevent applications from automatically following hard links.

4. Verify File Ownership

Before accessing a file, check the file owner and permissions to ensure it belongs to a trusted source.

5. Avoid Using Shared Writable Directories

Do not store sensitive or critical files in publicly writable directories where attackers can create malicious hard links.

6. Regular Security Monitoring and Testing

Perform regular vulnerability scans and security audits to detect and fix hard link related vulnerabilities in the system.

7. Vulnmerability Name: Exposure of Sensitive Information to an Unauthorized Actor

CWE: CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

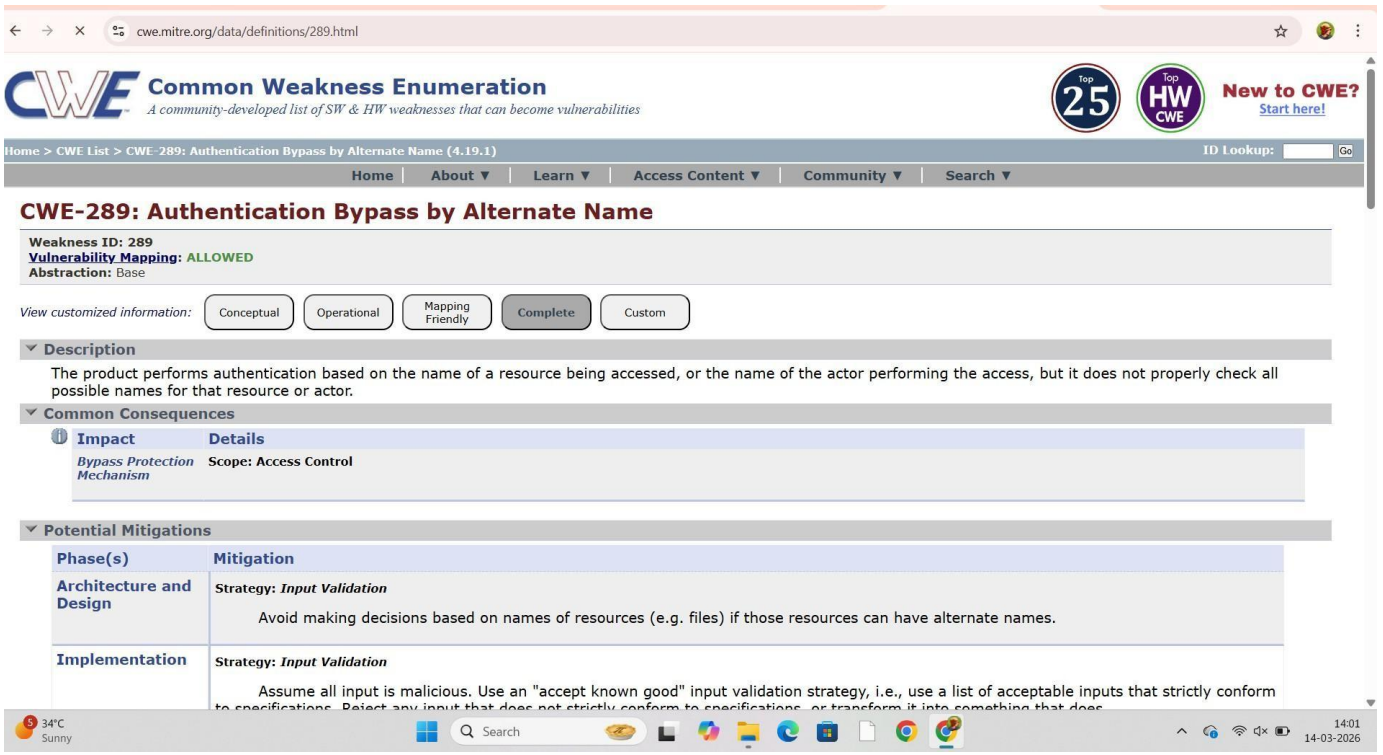
OWASP Category: A03:2021 – Sensitive Data Exposure

Description:

Exposure of Sensitive Information to an Unauthorized Actor occurs when confidential or sensitive data is accessed by individuals who do not have permission to view it. This can happen due to weak security controls, improper authentication, insecure storage, misconfigured servers, or lack of encryption. Sensitive information may include user credentials, personal data, financial records, business documents, or system configuration details. If proper protection mechanisms are not implemented, attackers or unauthorized users may gain access to this information.

Business Impact:

This vulnerability can cause significant damage to an organization. Unauthorized access to sensitive information may lead to data breaches, identity theft, and misuse of confidential data. It can result in financial losses, legal penalties, and non-compliance with data protection regulations. Additionally, such incidents can damage the organization's reputation, reduce customer trust, and negatively affect business operations and long-term growth



cwe.mitre.org/data/definitions/289.html

Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-289: Authentication Bypass by Alternate Name (4.19.1)

ID Lookup:

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-289: Authentication Bypass by Alternate Name

Weakness ID: 289
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product performs authentication based on the name of a resource being accessed, or the name of the actor performing the access, but it does not properly check all possible names for that resource or actor.

Common Consequences

Impact	Details
Bypass Protection Mechanism	**Scope: Access Control**

Potential Mitigations

Phase(s)	Mitigation
Architecture and Design	**Strategy: *Input Validation*** Avoid making decisions based on names of resources (e.g. files) if those resources can have alternate names.
Implementation	**Strategy: *Input Validation*** Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use a list of acceptable inputs that strictly conform

34°C Sunny | Search | 14:01 14-03-2026

Recommendation/Mitigation:

1. **Implement Strong Access Control**

Ensure that only authorized users can access sensitive information by using proper authentication and authorization mechanisms.

2. **Encrypt Sensitive Data**

Use strong encryption to protect sensitive information both in storage (at rest) and during transmission (in transit).

3. **Apply the Principle of Least Privilege**

Grant users only the minimum level of access required to perform their tasks to reduce the risk of unauthorized data exposure.

4. **Secure Configuration of Systems**

Properly configure servers, databases, and applications to prevent accidental exposure of confidential information.

5. **Regular Security Monitoring**

Continuously monitor system logs and user activities to detect and respond to unauthorized access attempts.

6. **Perform Security Testing**

Conduct regular vulnerability scans, penetration testing, and security audits to identify and fix weaknesses.

7. **Data Masking and Protection**

Use techniques such as data masking or tokenization to hide sensitive data when it is displayed or processed.

8. **Vulnerability Name:** Exposure of Sensitive Information Through Sent Data

CWE: CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

OWASP Category: A03:2021- Sensitive Data Exposure

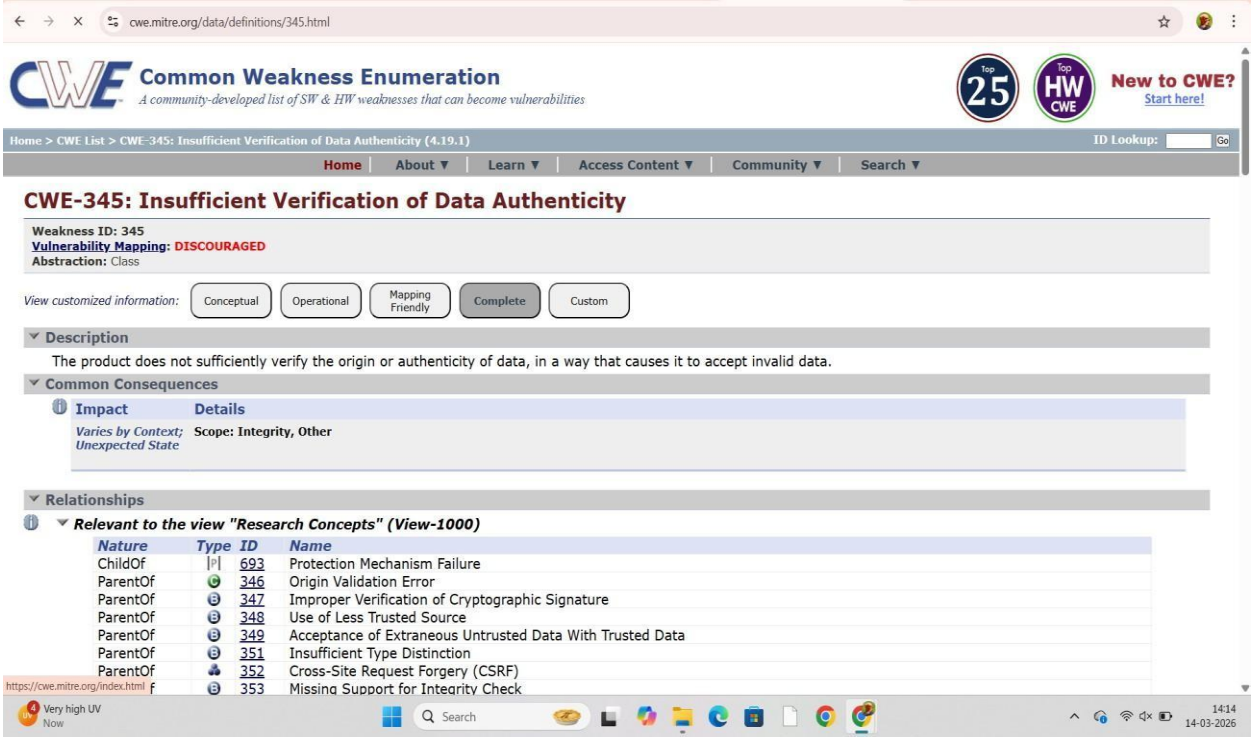
Description:

Exposure of Sensitive Information Through Sent Data occurs when an application sends confidential or sensitive data over a network without proper protection. This can happen if data such as passwords, personal information, financial data, API keys, or session tokens are transmitted without encryption (e.g., using HTTP instead of HTTPS) or are included in URLs, logs, or request parameters. Attackers who intercept network traffic (through techniques like packet sniffing or man-in-the-middle attacks) may capture this sensitive information. Poor security practices such as sending plaintext data, improper API responses, or insecure communication protocols can lead to this vulnerability.

Business Impact:

- **Data Breaches**
  Sensitive information such as personal data, login credentials, or financial details may be accessed by unauthorized users, leading to serious data breaches.
- **Financial Loss**
  Organizations may suffer financial losses due to fraud, legal penalties, compensation to affected users, and costs for fixing the security issue.
- **Loss of Customer Trust**
  When sensitive data is exposed, customers may lose confidence in the organization's ability to protect their information.

- **Legal and Compliance Issues**
  Exposure of confidential data may violate data protection laws and regulations, resulting in legal actions or regulatory fines.
- **Reputation Damage**
  Security incidents can harm the organization's reputation and negatively affect business relationships and future opportunities.



- **Recommendation/Mitigation:**
- **Use Secure Communication Protocols**
  Always transmit sensitive information using HTTPS with SSL/TLS encryption instead of HTTP to protect data during transmission.
- **Encrypt Sensitive Data**
  Apply strong encryption methods for sensitive information such as passwords, personal data, and financial details before sending it over the network.
- **Avoid Sending Sensitive Data in URLs**
  Do not include confidential information in URL parameters, query strings, or GET requests, as they can be logged or stored in browser history.
- **Implement Secure Authentication Mechanisms**
  Use secure authentication methods such as token-based authentication, OAuth, or multi-factor authentication (MFA) to protect user data.
- **Use Secure Session Management**
  Protect session IDs and tokens using secure cookies and proper session expiration policies.

- **Limit Data Exposure**
  Only send the minimum required information in responses or API communications to reduce the risk of data leakage.
- **Enable Proper Logging and Monitoring**
  Monitor network traffic and application logs to detect unauthorized access or suspicious data transmissions.
- **Regular Security Testing**
  Perform vulnerability scanning, penetration testing, and code reviews to identify and fix data exposure issues.

9. Vulnerability Name: Storage of File with Sensitive Data Under Web Root

CWE: CWE-532: Insertion of Sensitive Information into Log File or Web Accessible Location

OWASP Category: A03: 2021- Sensitive Data Exposure

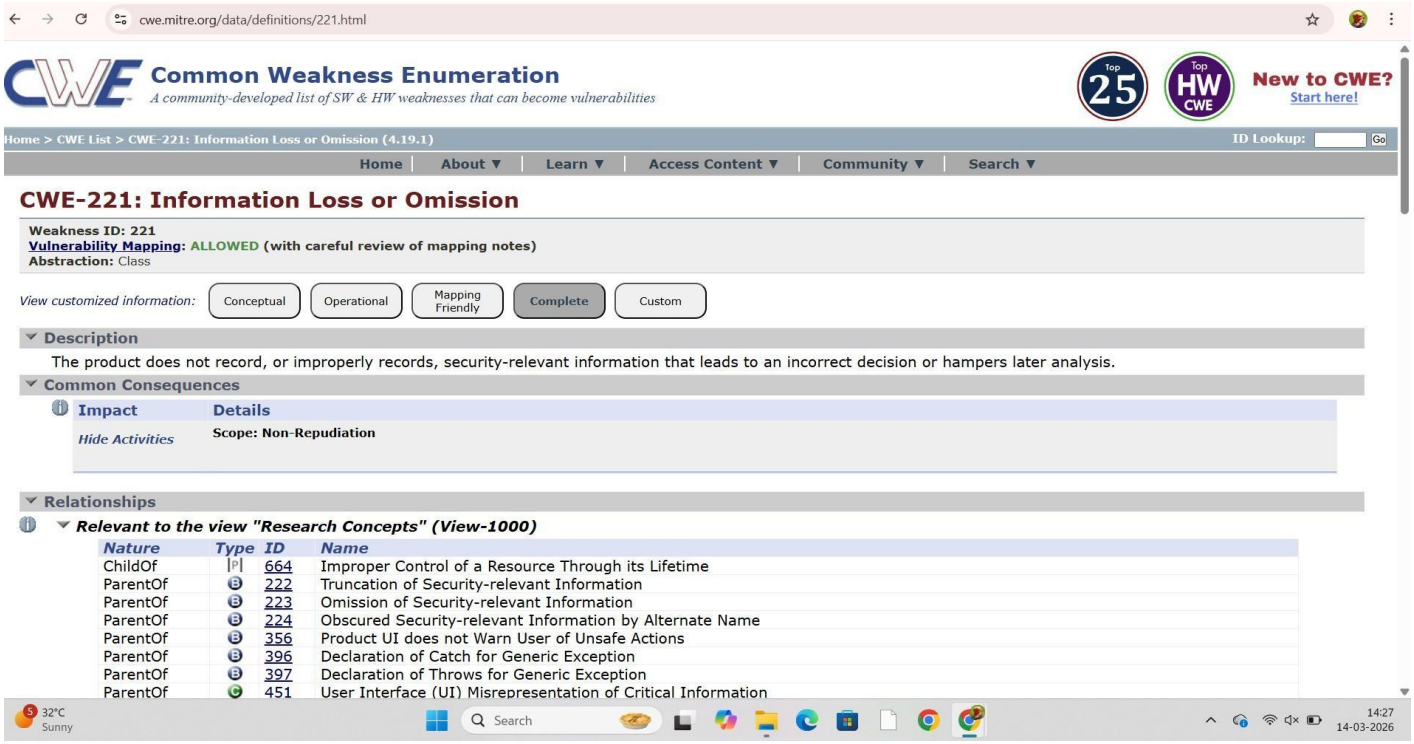
Description:

Storage of File with Sensitive Data Under Web Root occurs when confidential or sensitive files are stored in a directory that is publicly accessible through the web server (web root). These files may include configuration files, backup files, logs, database exports, source code, or documents containing sensitive information. If these files are placed inside the web root without proper access restrictions, attackers can directly access them using a web browser or automated tools. This may expose sensitive data such as credentials, API keys, personal information, or system configuration details, which can then be exploited for further attacks.

Business Impact:

- **Unauthorized Data Access:** Attackers may access confidential files containing sensitive business or customer information.
- **Credential Exposure:** Database passwords, API keys, or system credentials stored in files may be compromised.
- **Data Breach Risk:** Exposure of personal or financial data can lead to major data breaches.
- **System Compromise:** Attackers may use exposed configuration or source code to identify vulnerabilities and exploit the system.

- **Reputation Damage and Compliance Issues:** Organizations may face loss of customer trust, legal penalties, and regulatory compliance violations due to data exposure.



Recommendation/Mitigation:

- **Store Sensitive Files Outside the Web Root**

  Keep confidential files such as configuration files, backups, and database exports outside the publicly accessible web directory.

- **Implement Access Control**

  Restrict access to sensitive files using **p**roper authentication, authorization, and file permission settings.

- **Disable Directory Listing**

  Configure the web server to disable directory browsing so attackers cannot view or download files from directories.

- **Encrypt Sensitive Data**

  Sensitive information stored in files should be encrypted to prevent misuse even if accessed.

- **Remove Unnecessary Files**

  Delete backup files, temporary files, and old configuration files that are no longer required.

- **Use Secure File Permissions**
  Set least privilege permissions so that only authorized users or applications can access sensitive files.

10. Vulnerabilty Name: Incorrect Default Permission

CWE: CWE-276: Incorrect Default Permission

OWASP Category: A05:2021-Security Misconfiguration

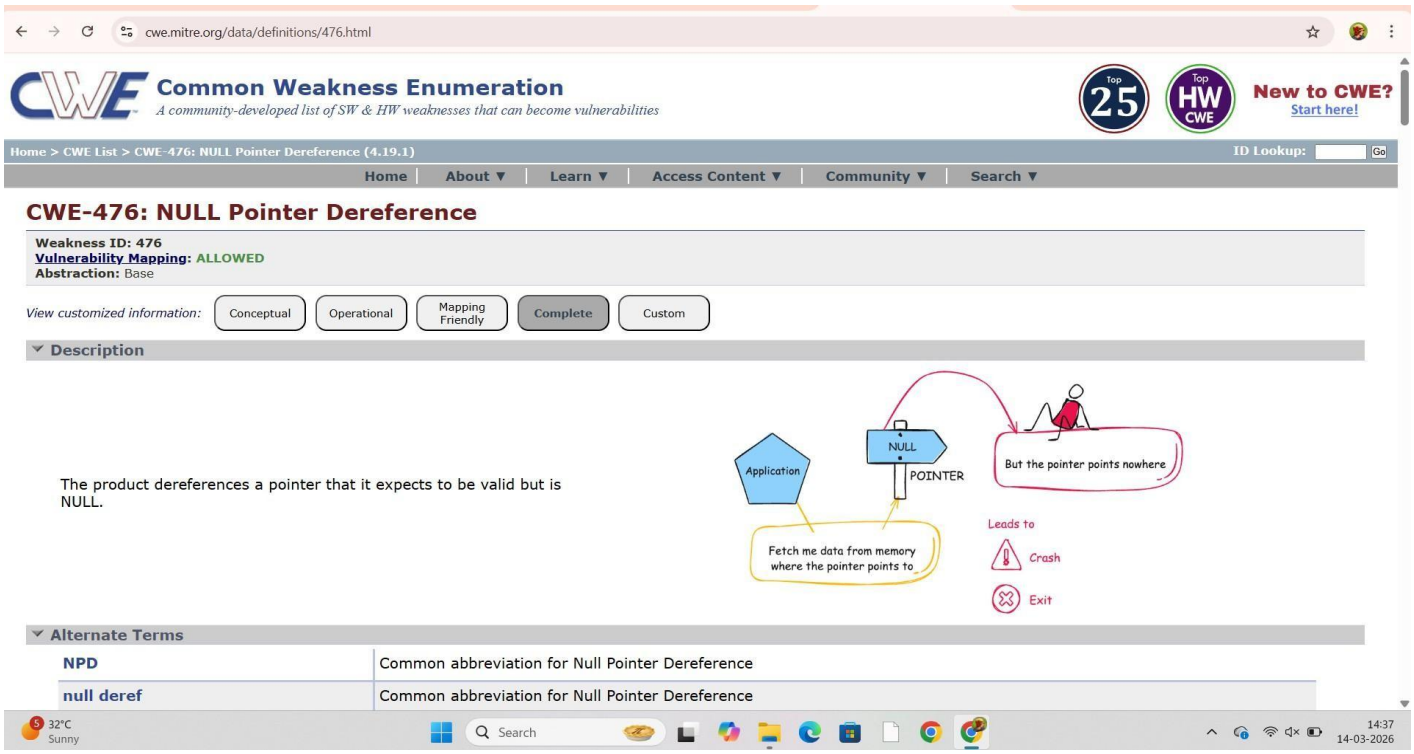
Description:

Incorrect Default Permission occurs when files, directories, or system resources are created with **insecure or overly permissive default access rights**. Instead of restricting access to only authorized users, the system allows **unnecessary read, write, or execute permissions** to other users or applications.

This vulnerability usually happens due to **misconfigured operating systems, applications, or servers** where default settings are not properly modified after installation. As a result, unauthorized users may gain access to sensitive files, modify system data, or execute malicious actions.

Business Impact:

- **Unauthorized Access:** Attackers or unauthorized users may access confidential files or system resources.
- **Data Modification:** Sensitive data can be altered, deleted, or corrupted by unauthorized individuals.
- **System Compromise:** Excessive permissions may allow attackers to execute malicious code or gain higher privileges.
- **Data Leakage:** Confidential business or customer information may be exposed.
- **Operational and Reputation Damage:** Security incidents caused by improper permissions can lead to financial loss and damage to the organization's reputation.



Recommendation/Mitigation:

- **Apply the Principle of Least Privilege**
  Assign only the **minimum permissions required** for users and applications to perform their tasks.
- **Review and Modify Default Permissions**
  After installing applications or systems, **check and adjust default file and directory permissions** to ensure they are secure.
- **Use Proper Access Control Mechanisms**
  Implement **role-based access control (RBAC)** or similar mechanisms to restrict access to authorized users only.
- **Regular Permission Audits**
  Conduct **periodic security audits** to review and correct unnecessary or excessive permissions.
- **Secure Configuration Management**
  Follow **secure configuration guidelines** for operating systems, databases, and applications to avoid insecure default settings.

- **Monitor and Log Access Activities**

  Enable **logging and monitoring** to detect unauthorized access or suspicious activities related to file permissions.

Stage- 2

Overview of Zscaler Private Access

Zscaler Private Access (ZPA) is a cloud-based security solution developed by **Zscaler** that provides secure access to internal applications without exposing the network to the internet. It follows the **Zero Trust Security Model**, which means **no user or device is trusted by default**, and access is granted only after proper authentication and verification.

ZPA allows employees, partners, or remote users to securely connect to private applications located in data centers or cloud environments. Instead of providing full network access like traditional VPNs, ZPA gives **application-level access**, which improves security and reduces the risk of cyberattacks.

Key Features

- **Zero Trust Architecture:** Every user and device must be verified before accessing applications.
- **Application-Level Access:** Users can only access specific applications, not the entire network.
- **Secure Remote Access:** Enables employees to securely access internal systems from anywhere.
- **Cloud-Based Security:** The service runs in the cloud, reducing the need for traditional VPN infrastructure.
- **Identity-Aware Access Policies:** Access is controlled based on user identity, device posture, and security policies.

Benefits

- Protects internal applications from unauthorized access
- Reduces the risk of network attacks and data breaches
- Improves security for remote and hybrid work environments
- Eliminates the limitations of traditional VPN solutions

About Nessus:

Nessus is a popular **vulnerability assessment tool** developed by **Tenable**. It is used by cybersecurity professionals to **identify security vulnerabilities, misconfigurations, and missing patches** in computers, servers, networks, and applications.

Nessus scans systems and compares them with a large database of known security vulnerabilities. After scanning, it generates detailed **security reports** that show the detected vulnerabilities, their severity levels, and recommended solutions to fix them.

Key Features

- **Vulnerability Scanning:** Detects security weaknesses in systems and applications.
- **Configuration Checking:** Identifies incorrect security configurations.
- **Patch Detection:** Finds missing software updates and patches.
- **Risk Assessment:** Classifies vulnerabilities based on severity (low, medium, high, critical).
- **Detailed Reports:** Provides remediation steps to fix the identified issues.

Uses

- Network security assessment
- Application vulnerability scanning
- Compliance checking
- Security auditing and risk management

Target Details:

Target Website: https://public-firing-range.appspot.com

Target Website Address: 142.251.39.148

List of Vulnerability Table:

S.No	Vulnerability Name	Severity	Plug-In
1.	SSL(Multiple Issues)	Mixed	10863
2.	TLS(Multiple Issues)	Mixed	42873
3.	HTTP(Multiple Issues)	Info	11219
4.	TLS(Multiple Issues)	Info	56984
5.	Service Detection	Info	22964
6.	Nessus Synchronize Scanner	Info	19506
7.	Device Type	Info	54615
8.	Hostfully Qualified Domain	Info	12053
9.	Nessus Scan Information	Info	19506
10.	Open SSL Detection	Info	50845

Report

Vulnerability Name: SSL (Multiple Issues)

Severity: Mixed

Plug-in: 10863

Port: 443

Description

SSL (Secure Sockets Layer) Multiple Issues refers to a situation where several security weaknesses are detected in the SSL/TLS configuration of a server. These issues may include the use of outdated SSL/TLS protocols, weak encryption algorithms, improper certificate configurations, or insecure cipher suites.

Attackers may exploit these weaknesses to intercept, decrypt, or manipulate sensitive data transmitted between the client and the server. Common issues detected in vulnerability scanners like Nessus include weak ciphers, expired certificates, support for deprecated protocols such as SSL 2.0 or SSL 3.0, and improper TLS configurations.

These vulnerabilities weaken the secure communication channel that protects sensitive data such as login credentials, financial information, and personal data.

Business Impact

If SSL multiple issues are not addressed, the organization may face several risks:

- **Data Interception:** Attackers may perform Man-in-the-Middle (MITM) attacks and capture sensitive information transmitted over the network.
- **Loss of Confidentiality:** Sensitive data such as passwords, financial details, or customer information may be exposed.
- **Compliance Violations:** Weak SSL configurations may violate security standards such as PCI-DSS or ISO 27001.
- **Reputation Damage:** Customers may lose trust if the organization's website or services are considered insecure.
- **Unauthorized Access:** Weak encryption may allow attackers to decrypt traffic and gain access to protected systems.

Solution

To mitigate SSL multiple issues, organizations should follow secure SSL/TLS configuration practices:

- **Disable outdated protocols** such as SSL 2.0 and SSL 3.0.
- **Use strong TLS versions** like TLS 1.2 or TLS 1.3.
- **Remove weak cipher suites** and enable only strong encryption algorithms.
- **Use valid and trusted SSL certificates** issued by a trusted Certificate Authority.
- **Regularly update the server and cryptographic libraries** such as OpenSSL.
- **Perform periodic vulnerability scanning** using tools like Nessus to detect SSL misconfigurations.

Vulnerability Name: TLS (Multiple Issues)

Severity: Mixed

Plug-in: 42873

Port: 443

Description

TLS (Transport Layer Security) Multiple Issues refers to several security weaknesses found in the TLS configuration of a server. TLS is used to secure communication between clients and servers by encrypting transmitted data.

When a system supports outdated TLS versions, weak cipher suites, or misconfigured certificates, attackers may exploit these weaknesses to intercept or manipulate secure communication. Vulnerability scanners like Nessus detect these problems during security assessments.

Common TLS issues include the use of deprecated versions such as TLS 1.0 and TLS 1.1, weak encryption algorithms, or improper certificate validation. These weaknesses can compromise the confidentiality and integrity of data transmitted between systems.

Business Impact

TLS configuration issues can create significant security risks for organizations:

- **Data Interception:** Attackers may capture sensitive data through Man-in-the-Middle attacks.
- **Loss of Data Confidentiality:** Information such as login credentials, personal data, and financial details may be exposed.
- **Compliance Issues:** Weak TLS configurations may violate security standards like PCI-DSS.
- **System Compromise:** Attackers may exploit TLS weaknesses to gain unauthorized access to systems.
- **Reputation Damage:** Customers may lose trust if secure communication channels are vulnerable.

Solution

Organizations should follow secure TLS configuration practices to mitigate these issues:

- **Disable outdated TLS versions** such as TLS 1.0 and TLS 1.1.
- **Use secure versions** like TLS 1.2 or TLS 1.3.
- **Remove weak cipher suites** and enable strong encryption algorithms.
- **Use valid SSL/TLS certificates** from trusted certificate authorities.
- **Regularly update cryptographic libraries** such as OpenSSL.
- **Conduct regular vulnerability scanning** using tools like Nessus to identify and fix TLS issues.

Vulnerability Name: HTTP(Multiple Issues)

Severity: Info

Plug-in: 11219

Port: 443

Description

HTTP (Multiple Issues) indicates that several security weaknesses have been detected in the HTTP service running on a server. HTTP is used to transfer web data between clients and servers, but improper configuration can expose sensitive information and system details.

Common HTTP issues include insecure headers, information disclosure, directory listing, outdated web server versions, and insecure HTTP methods (such as PUT, DELETE, or TRACE). These weaknesses may allow attackers to gather information about the system or exploit the web application.

Security scanners such as Nessus identify these problems during vulnerability assessments by analyzing HTTP responses and server configurations.

Business Impact

HTTP configuration issues can create several risks for organizations:

- **Information Disclosure:** Attackers may obtain server details such as software versions or internal configurations.
- **Unauthorized Access:** Misconfigured HTTP methods may allow attackers to upload or modify files.
- **Web Application Attacks:** Vulnerabilities may assist attackers in launching further attacks like cross-site scripting or injection attacks.
- **Data Exposure:** Sensitive information transmitted over unsecured HTTP may be intercepted.
- **Reputation Damage:** Insecure web services may reduce user trust in the organization.

Solution

To reduce HTTP-related vulnerabilities, organizations should implement the following security measures:

- **Disable unnecessary HTTP methods** such as TRACE, PUT, or DELETE if not required.
- **Use secure HTTP headers** like Content Security Policy, X-Frame-Options, and X-Content-Type-Options.
- **Hide server version information** to prevent information disclosure.
- **Update and patch the web server software** regularly.
- **Redirect HTTP traffic to HTTPS** to ensure encrypted communication using Transport Layer Security.
- **Perform regular vulnerability scans** using tools such as Nessus.

Vulnerability Name: TLS (Multiple Issues)

Severity: Mixed

Plug-in: 42873

Port: 443

Description

TLS (Transport Layer Security) Multiple Issues refers to several security weaknesses found in the TLS configuration of a server. TLS is used to secure communication between clients and servers by encrypting transmitted data.

When a system supports outdated TLS versions, weak cipher suites, or misconfigured certificates, attackers may exploit these weaknesses to intercept or manipulate secure communication. Vulnerability scanners like Nessus detect these problems during security assessments.

Common TLS issues include the use of deprecated versions such as TLS 1.0 and TLS 1.1, weak encryption algorithms, or improper certificate validation. These weaknesses can compromise the confidentiality and integrity of data transmitted between systems.

Business Impact

TLS configuration issues can create significant security risks for organizations:

- **Data Interception:** Attackers may capture sensitive data through Man-in-the-Middle attacks.
- **Loss of Data Confidentiality:** Information such as login credentials, personal data, and financial details may be exposed.
- **Compliance Issues:** Weak TLS configurations may violate security standards like PCI-DSS.
- **System Compromise:** Attackers may exploit TLS weaknesses to gain unauthorized access to systems.
- **Reputation Damage:** Customers may lose trust if secure communication channels are vulnerable.

Solution

Organizations should follow secure TLS configuration practices to mitigate these issues:

- **Disable outdated TLS versions** such as TLS 1.0 and TLS 1.1.
- **Use secure versions** like TLS 1.2 or TLS 1.3.
- **Remove weak cipher suites** and enable strong encryption algorithms.
- **Use valid SSL/TLS certificates** from trusted certificate authorities.
- **Regularly update cryptographic libraries** such as OpenSSL.
- **Conduct regular vulnerability scanning** using tools like Nessus to identify and fix TLS issues.

Vulnerability Name: Service Detection

Severity: Info

Plug-in: 22964

Port: 80

Description

Service Detection refers to the process where a vulnerability scanner identifies the services running on a target system and the corresponding ports they are operating on. During a scan, tools like Nessus analyze open ports to determine the type of service, protocol, and sometimes the software version running on the system.

Although service detection itself is not always a vulnerability, exposing service information such as server versions, running applications, or network services may assist attackers in identifying potential targets. Attackers can use this information to discover known vulnerabilities associated with those services and launch further attacks.

Business Impact

If service information is exposed, it may create several security risks:

- **Information Disclosure:** Attackers may identify software versions and services running on the system.
- **Targeted Attacks:** Knowledge of services allows attackers to search for known vulnerabilities related to those services.
- **Increased Attack Surface:** Multiple exposed services increase the chances of exploitation.
- **Unauthorized Access:** Misconfigured or outdated services may allow attackers to gain access to systems.

Solution

To reduce the risk associated with service detection, organizations should follow these security practices:

- **Disable unnecessary services** that are not required for system operations.
- **Close unused ports** on servers and network devices.
- **Apply regular security patches and updates** to all running services.
- **Use firewall rules** to restrict access to critical services.
- **Hide or limit service version information** to prevent attackers from identifying system details.
- **Perform periodic vulnerability scans** using tools such as Nessus to monitor exposed services.

Vulnerability Name: Nessus Synchronize Scanner

Severity: Info

Plug-in: 19506

Port: 8834

Description

Nessus Synchronize Scanner refers to the process where the vulnerability scanner synchronizes its plugins, vulnerability database, and configuration settings with the central server. This ensures that the scanner has the latest vulnerability definitions and security checks before performing a scan.

During synchronization, the scanner communicates with the update servers to download new plugins and vulnerability information. If this communication is not properly secured or restricted, it may expose information about the scanning infrastructure. Security tools such as Nessus perform this synchronization regularly to keep the scanning engine updated with newly discovered vulnerabilities.

Business Impact

Improper configuration or exposure of the scanner synchronization process may lead to several risks:

- **Information Disclosure:** Attackers may identify the presence of a vulnerability scanning system in the network.
- **Scanner Targeting:** If attackers know the scanning infrastructure, they may attempt to evade or manipulate scans.
- **Unauthorized Access:** Misconfigured scanner services may allow unauthorized users to access scanner information.
- **Security Monitoring Disruption:** If synchronization fails or is tampered with, the scanner may miss newly discovered vulnerabilities.

Solution

Organizations should secure the scanner synchronization process by following these practices:

- **Restrict access** to the vulnerability scanner management interface.
- **Allow synchronization only with trusted update servers.**
- **Use secure communication channels** when downloading updates.
- **Regularly update the scanner software** to maintain security and functionality.
- **Monitor scanner logs** to detect unauthorized access attempts.
- **Implement firewall rules** to limit access to the scanner system.

Regular updates and monitoring of tools such as Nessus help ensure accurate vulnerability detection and secure scanning operations.

Vulnerability Name: Device Type

Severity: Info

Plug-in: 54615

Port: 22

Description

Device Type refers to the identification of the type of device detected during a vulnerability scan. The scanning tool analyzes open ports, services, operating systems, and network responses to determine whether the target system is a server, workstation, router, firewall, printer, or other network device.

This information helps security administrators understand the assets present in the network and apply appropriate security controls. Tools such as Nessus use service detection and operating system fingerprinting techniques to classify the device type.

Although device type detection itself is not a vulnerability, exposing device information may help attackers understand the network environment and identify potential targets.

Business Impact

If device information is exposed, it may lead to several security risks:

- **Network Reconnaissance:** Attackers may identify critical devices such as servers, firewalls, or databases.
- **Targeted Attacks:** Knowing the device type helps attackers search for known vulnerabilities related to that device.
- **Increased Attack Surface:** Attackers may focus on high-value systems such as application servers or database servers.
- **Information Disclosure:** Network infrastructure details may become visible to unauthorized users.

Solution

Organizations should reduce exposure of device information by implementing the following security practices:

- **Limit network scanning access** using firewall rules.
- **Disable unnecessary services and ports** on network devices.
- **Apply regular security patches and updates** to systems.
- **Use network segmentation** to protect critical devices.
- **Restrict system information disclosure** wherever possible.
- **Monitor network activity** to detect unauthorized scanning attempts.

Security teams often use tools such as Nessus to identify device types and ensure appropriate security configurations.

Vulnerability Name: Hostfully Qualified Domain

Severity: Info

Plug-in: 12053

Port: 53

Description

A Fully Qualified Domain Name (FQDN) is the complete domain name that uniquely identifies a host within the Domain Name System (DNS). It includes the hostname and the domain name, allowing systems to locate and communicate with each other on a network or the internet.

During vulnerability scans, tools such as Nessus identify the host's FQDN through DNS lookups and network responses. This helps administrators verify the target system being scanned and associate vulnerabilities with the correct host.

Although FQDN identification itself is not a vulnerability, exposing internal domain names may provide useful information to attackers during the reconnaissance phase.

Business Impact

Exposure of the host fully qualified domain name may lead to the following risks:

- **Information Disclosure:** Attackers may obtain internal domain structure and hostnames.
- **Network Reconnaissance:** Knowledge of domain names may help attackers map the network environment.
- **Targeted Attacks:** Attackers can identify specific servers such as mail servers, web servers, or database servers.
- **Security Enumeration:** Revealed domain details may assist attackers in further scanning and exploitation.

Solution

Organizations should minimize exposure of domain information through the following measures:

- **Restrict public DNS records** to only necessary systems.
- **Hide internal hostnames** from external networks.
- **Implement firewall rules** to control DNS queries.
- **Use network segmentation** to protect internal systems.
- **Monitor DNS logs** for suspicious or unauthorized queries.
- **Perform regular security scans** using tools like Nessus to identify exposed host information

Vulnerability Name: Open SSL Detection

Severity: Info

Plug-in: 50845

Port: 443

Description

OpenSSL Detection indicates that the scanning tool has identified the presence of the OpenSSL cryptographic library on the target system. OpenSSL is widely used to implement secure communication protocols such as SSL and TLS for encrypting data transmitted between clients and servers.

During vulnerability scans, tools like Nessus analyze network services to determine whether they are using the OpenSSL library for encryption. This detection helps security administrators identify systems that rely on OpenSSL for secure communications.

While the detection itself is informational, outdated or vulnerable versions of OpenSSL may expose systems to known security vulnerabilities that could be exploited by attackers.

Business Impact

If vulnerable versions of OpenSSL are used, organizations may face several security risks:

- **Encryption Weakness:** Outdated cryptographic libraries may contain vulnerabilities affecting secure communication.
- **Data Exposure:** Attackers may exploit vulnerabilities to access sensitive information transmitted over encrypted channels.
- **Man-in-the-Middle Attacks:** Weak cryptographic implementations may allow attackers to intercept and manipulate encrypted traffic.
- **Compliance Issues:** Using outdated security libraries may violate security compliance standards.

Solution

To reduce risks associated with OpenSSL vulnerabilities, organizations should follow these security practices:

- **Update OpenSSL to the latest stable version** to patch known vulnerabilities.
- **Disable weak SSL/TLS protocols** and enable strong encryption protocols.
- **Regularly apply security patches** to servers and applications that use OpenSSL.
- **Monitor vulnerability advisories** related to OpenSSL.
- **Exposure to Known Vulnerabilities:** If the detected version of OpenSSL is outdated, it may contain publicly known vulnerabilities that attackers can exploit.
- **Unauthorized Data Access:** Weak encryption implementations may allow attackers to decrypt sensitive information such as user credentials, financial data, or confidential communications.
- **Man-in-the-Middle (MITM) Attacks:** Attackers may intercept and alter encrypted communication between users and servers using weaknesses in SSL/TLS configurations.
- **System Compromise:** Exploiting OpenSSL vulnerabilities could allow attackers to execute malicious code or gain unauthorized access to the server.
- **Loss of Confidentiality and Integrity:** Sensitive data transmitted over encrypted channels may be exposed or modified by attackers.

Stage-3

Report

Title:APPLICATION SPECIFIC ACCESS POLICIES USING ZSCALER PRIVATE ACCESS

Security Operations Center (SOC)

A Security Operations Center (SOC) is a dedicated team that manages and monitors the security of an organization's network and systems. It helps detect cyber threats, analyze suspicious activities, and protect sensitive data. In this project, SOC plays an important role in monitoring user access to applications secured by Zscaler Private Access. Instead of using traditional VPNs, Zscaler allows secure and controlled access to applications, and SOC teams monitor these activities to ensure only authorized users can connect.

SOC Cycle

The SOC cycle represents the process followed to manage security threats. It begins with monitoring network activity and collecting security data. When abnormal behavior is detected, the security team analyzes the data to identify potential threats. After investigation, appropriate actions are taken to stop the attack and secure the system. Finally, systems are restored and preventive steps are implemented to avoid future incidents.

Security Information and Event Management (SIEM)

Security Information and Event Management (SIEM) is a system used to collect and analyze security data from multiple sources such as servers, applications, and network devices. It helps organizations understand what is happening in their network in real time. In this project, SIEM tools monitor authentication attempts and access logs generated when users connect to applications through Zscaler Private Access.

SIEM Cycle

The SIEM cycle involves several steps such as log collection, event analysis, correlation of data, and alert generation. Security logs from different systems are gathered and processed. These logs are then compared to identify unusual activities. If the system detects suspicious behavior, alerts are sent to the security team so they can investigate and take necessary actions.

Malware Information Sharing Platform (MISP)

MISP is an open-source platform used for sharing threat intelligence among organizations. It helps security teams learn about new malware attacks, vulnerabilities, and malicious IP addresses. By sharing threat data, organizations can improve their cybersecurity defenses and respond faster to potential attacks.

Your College Network Information

The college network includes various digital resources such as computers, servers, wireless networks, and internet connections used by students and faculty members. These systems allow users to access online learning platforms, institutional databases, and internal applications. Because many users access the network daily, strong security controls are necessary to protect sensitive information and maintain network stability.

How You Think You Deploy SOC in Your College

To deploy SOC in the college environment, security monitoring tools can be installed to collect data from network devices, servers, and authentication systems. These logs can be analyzed using SIEM platforms to detect unusual behavior. By implementing Zscaler Private Access, the college can create application-specific access policies so that only verified users can reach internal applications. SOC analysts can continuously monitor this access and respond to potential threats quickly.

Threat Intelligence

Threat intelligence refers to the knowledge gathered about potential cyber threats and attacker behavior. This information helps security teams understand how attacks occur and how they can be prevented. By studying threat intelligence data, organizations can improve their security strategies and protect systems from emerging cyber risks.

Incident Response

Incident response is the structured approach used to handle cybersecurity incidents. When a threat is discovered, the security team analyzes the problem, isolates affected systems, and removes the malicious activity. After resolving the issue, systems are restored and measures are taken to prevent similar incidents in the future.

QRadar & Understanding About Tool IBM

QRadar is a powerful security monitoring tool used for threat detection and analysis. It gathers security logs from multiple sources and processes them to identify suspicious activities. QRadar provides a visual dashboard that allows security analysts to track events, investigate threats, and manage security incidents effectively.

Conclusion

Stage 1

Stage 1 helped in understanding web application security testing and identifying vulnerabilities that may affect application performance and safety.

Stage 2

Stage 2 focused on analyzing vulnerability scan results generated using Nessus, which helps identify weaknesses in systems and networks.

Stage 3

Stage 3 provided knowledge about cybersecurity monitoring technologies such as SOC, SIEM, and QRadar. These tools help detect threats, monitor network activity, and improve overall system security.

Future Scope :

Stage 1

Future improvements in web application testing can include automated vulnerability scanning and continuous security monitoring.

Stage 2

The testing process can be enhanced by integrating advanced vulnerability assessment tools that provide faster and more accurate security analysis.

Stage 3

Future developments in SOC and SIEM may include the use of Artificial Intelligence and Machine Learning to detect threats faster and automate incident response processes.

Topics Explored :

- Cloud Security
- Identity and Access Management
- Security Operations Center
- SIEM Analysis
- Threat Intelligence
- Incident Response

Tools Explored :

- Zscaler private Access
- Microsoft Azure Active Directory
- IBM QRadar
- MISP Platform

THE END